

1. **First Round - to be scheduled the same way you scheduled with me. Select time on interviewer's calendar and let me know if you do not see times that work. EMAIL me the next day so I can make sure I get your feedback.**
 1. Technical coding interview live with engineer on HackerRank.com:
 2. Act not iterate
2. **Final Interview – 4 rounds.** Once scheduled, I will send you the assigned competencies (OCI Values, Technical and/or role related competencies)
 1. 2 are engineers or potential colleagues and will be a mix of technical (Design problems, coding challenges, technical discussions) and behavioral
 2. 1 is the hiring manager (technical & behavioral)
 3. 1 is an outside party (called the key interviewer) based interview questions but usually has a technical background. Also serves as the mediator on your candidate debrief
3. **Debrief** within 48 hours
4. **Offer**

Please reach out to me within 48 hours if you have not heard from us on interview feedback. Also, let me know if you have not been given the competencies for your final interview.

Tech Guidance – Concepts, resources, best practices.

See attached word document.

Behavioral Guidance (see attached document on OCI Core Values)

This will be about how you present your experience and the value you bring. Past success and behavior in this case is important. They will be looking for things you have already done to indicate how you will perform once hired. They will evaluate you using behavioral based interview questions which are often expressed in “what if” or “tell me about a time” type scenarios. When responding to these types of questions, it is best to answer using the following acronyms and to draw from your actual experience even when they propose a “what if.”

PAR – Problem, Action, Result

Or

STAR – Situation/Task, Action Result

- Here was the situation, task or problem (explain the situation and the need)
- Here was the action I took to correct or fix and why I decided to pursue this course
- Here was the end result- learning, success, growth; demonstrate value.

You are telling a story so be engaging and excited about it.

Think about specific experiences, projects, events, you feel will demonstrate the most value. Challenges (challenging people included), obstacles, failures, wins, learning experiences, etc. Refer to the Core Values for OCI for additional context and ideas on what to refer to. They will ask question directly related to these values. PRACTICE saying it using the above PAR or STAR format. Thank you

Interview Prep Guidance – Control Plane

Links/Resources for Prep:

- **Coding Practice:**
<https://www.hackerrank.com/dashboard> (listed in our most recent OCI Tech Interview Prep)
<https://edabit.com/challenges> (note on the side you can pick the language and level of difficulty)
<https://leetcode.com/explore/> (not only has coding exercises but also data structure/algorithms, system design, interview prep at companies like Amazon, MS, Google, FB. Sorry no Oracle/OCI)

List of other sites that candidates can explore for other SDE interview prep:
<https://www.zdnet.com/education/computers-tech/practice-coding-problems/>

- **Book:** Cracking the coding interview, Elements of programming interviews
- **System design :** [Highscalability.com](https://highscalability.com) – highly informative and extremely useful

Common Topics to focus on:

Algorithms and Data Structures:

- Various searching and sorting algorithms along with their time and space complexity – average, best and worst case
- Hash tables
- Binary Tree / n-array tree/tries
- Heaps
- Linked Lists
- Stacks/Queues
- Complexity analysis for look up/insertion/deletion into various data structures
- Recursion vs iterative programming

Operating Systems and Container basics:

- Threads

- Memory Organization/management.
- Program execution basics.
- Container basics. Running in VMs vs running in containers.
- Linux fundamentals. Knowledge of basic commands like grep, awk, uniq etc.

System Design

- Horizontal vs vertical scaling. Pros and Cons
- Availability, scalability, caching, load balancers, exponential backoff, two phase commits.
- Read up on how some of the more popular services like Twitter, Facebook live, Instagram are designed. Specifically focus on what kind of challenges they faced and how they overcame it.
- Be prepared to define SLAs for your service
 - Latency
 - Throughput
 - Availability
 - Durability
 - Security
 - Familiarize with following terms
 - P100, P99, P90, P50 etc
 - Blast Radius
 - System Invariant
 - Consensus, Leader election
 - Vector Clock
 - Eventual Consistency
 - High availability, Single point of failure

Approach

If it is coding problem

1. They will begin with some type of technical discussion more than likely, not sure on what
2. They will present some technical problems that you will need to solve.
 1. Example- what algorithm would you use to sort this table? There would be follow up questions about this algorithm.

3. Make sure you ask questions and qualify everything before you start to code. They are awarding you points per question and you will not get all the points if you do not ask qualifying questions.
4. If you do not know something, tell them that and tell them what resources you use when you are in this type of situation.
5. If you would reach out to a mentor or a senior engineer, treat this person that way. They will provide you guidance to see if you can follow instruction and figure it out.
6. The problems they give you, they may perceive to be out of your experience level. They want to help you. Ask questions, clarify what to do before you begin and think aloud for feedback as you go.
7. Clearly and meticulously discuss your solution before coding and factor in the time/space complexity of it as well.
8. Use the same approach for the test cases you run on your code as well

You are getting points for:

1. Qualifying question prior (quality of your questions)
2. Your solution
3. Your test cases